



7

Establishing Table Structures

*It is a capital mistake to theorize
before one has data.*

—SHERLOCK HOLMES,

THE ADVENTURES OF SHERLOCK HOLMES

Topics Covered in This Chapter

Defining the Preliminary Table List

Defining the Final Table List

Associating Fields with Each Table

Refining the Fields

Refining the Table Structures

Example: Establishing Table Structures

Summary

Review Questions

Organizations use databases to keep track of various subjects that are important to them. For example, a medical clinic keeps track of, among other things, its patients, doctors, and appointments; an equipment rental business must maintain data on its customers, equipment, and rental agreements; and a registrar's office is concerned (at the very least) with students, teaching staff, and courses. In every case—and in any other scenario you can imagine—a table within the database represents each subject. Furthermore, each table is composed of *fields*, which represent the characteristics that define or describe the subject of the table. Tables constitute the very foundation of the database, and they guarantee a solid and sound foundation when they are properly designed.

Defining the Preliminary Table List

During this portion of the database design process, you'll define a *Preliminary Table List* that you'll use to identify and establish the tables for the new database. You'll use three procedures to develop this list. The first involves using the *Preliminary Field List*, the second involves using the *list of subjects* you gathered during the interviewing process, and the third involves using the *mission objectives* you defined at the beginning of the database design process. You'll then move on to build the structure of each table using fields from the Preliminary Field List.

Identifying Implied Subjects

The process of defining the tables for the database begins with a review of the Preliminary Field List. Your objective is to identify subjects that are implied by the fields on the list.

You may wonder why you're reviewing the Preliminary Field List instead of starting with the list of subjects. The list of subjects does seem to be a more intuitive place to start. After all, you've carefully built this list during the interview process, and you've been influenced by the conversations you've had with the users and management. Surely, all of this has helped you identify every subject that needs to be represented in the database. You may be correct, but you could have a minor problem if you're wrong: missing tables.

Studying the fields on the primary field list helps you identify subjects from an *unbiased viewpoint*—you're letting the fields “talk” to you. It's crucial that you now look at this list as objectively as possible, as though you've never seen it before, *without* any of the biases you've assimilated during the interview process. This enables you to see how certain groups of fields suggest specific subjects, some of which may not have been identified during the interview process. You can also

use the Preliminary Field List to verify many of the subjects on the list of subjects. Using the Preliminary Field List in these ways allows you to cross-check your previous work and helps you ensure that the new database structure includes all the necessary subjects.

As you review the Preliminary Field List, ask yourself whether a certain set of fields defines or describes a particular subject. Move on to another set of fields if nothing readily comes to mind. When you can infer a subject from the field in the list, enter that subject on a new *Preliminary Table List*. Figure 7.1 shows a partial sample of a Preliminary Field List and illustrates how a set of fields can suggest a subject.

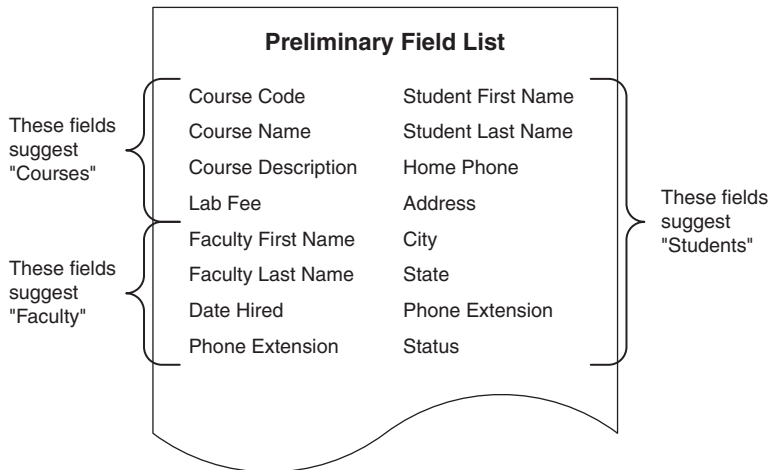


Figure 7.1 Using the Preliminary Field List to identify subjects.

Continue your review until you've scanned all the fields and identified as many subjects as possible. Be sure to add each subject you identify to the Preliminary Table List. This list will grow as you work with the list of subjects and mission objectives. Figure 7.2 shows an example of the first version of a Preliminary Table List.

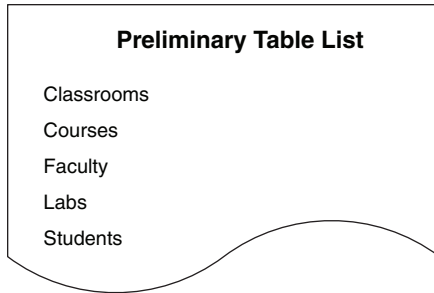


Figure 7.2 *The first version of the Preliminary Table List.*

Using the List of Subjects

Now, create a second version of the Preliminary Table List by merging the list of subjects (created during the interviews with users and management) with the first version of the Preliminary Table List (compiled by studying the Preliminary Field List). This new version contains a more complete list of tables. Merging the two lists is a three-step process, which involves resolving duplicate items, resolving items that represent the same subject, and combining the remaining items together into one list.

Step 1: Resolve Duplicate Items

Start this step by reviewing and cross-checking each item on the list of subjects against the items on the Preliminary Table List. Your objective here is to identify *duplicate items*, which are items on the list of subjects that already appear on the Preliminary Table List. You must be very careful how you resolve the duplicate items that you find. Begin by determining whether the items represent *different* subjects, despite the fact that they share the same name. (Use your interview notes as necessary to help you make your decision.) If they do represent different subjects, rename each item so that it accurately identifies the subject it represents and then add both items to the Preliminary Table List; otherwise, determine whether they truly represent the same

subject. When you conclude that both items do represent the same subject, cross out the item on the list of subjects and keep the one that appears on the Preliminary Table List. Resume the review until you've examined all the items on both the list of subjects and the Preliminary Table List. Let's take a look at an example of this process.

Assume that you're developing a database for an equipment rental business, and you're working with the list of subjects and the Preliminary Table List shown in Figure 7.3.

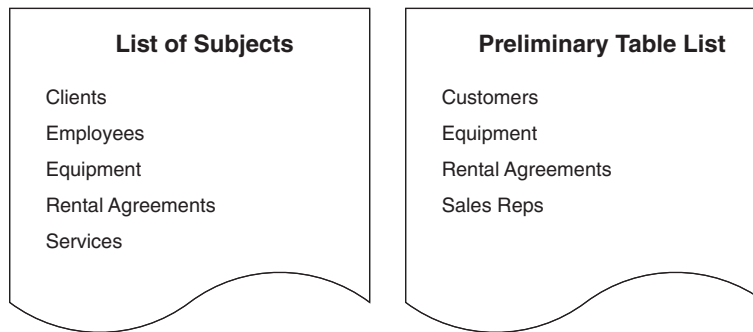


Figure 7.3 The list of subjects and the Preliminary Table List for an equipment rental business.

As you review these lists, you discover two duplicate items: “Equipment” and “Rental Agreements.” These items warrant further examination, so you start with “Equipment” and try to determine whether each occurrence represents a *different* subject. In reviewing your interview notes, you find that “Equipment” on the list of subjects represents items such as tools, appliances, and audiovisual equipment. You then remember that “Equipment” on the Preliminary Table List also includes trucks, vans, and trailers. You review your interview notes further and discover that vehicle rentals are treated differently from “regular” equipment rentals. Therefore, each occurrence of “Equipment” *does* represent a different subject. You resolve the duplication by keeping one occurrence of “Equipment” and renaming the other “Vehicles.” You then list both items on the Preliminary Table List.

Now you go through the same process with “Rental Agreements.” Fortunately, you discover that both occurrences share exactly the same meaning. The only thing you have to do in this case is cross out “Rental Agreements” on the list of subjects. Now you can continue your review until you’ve inspected each item on the list of subjects. Figure 7.4 shows the revised list of subjects and the Preliminary Table List.

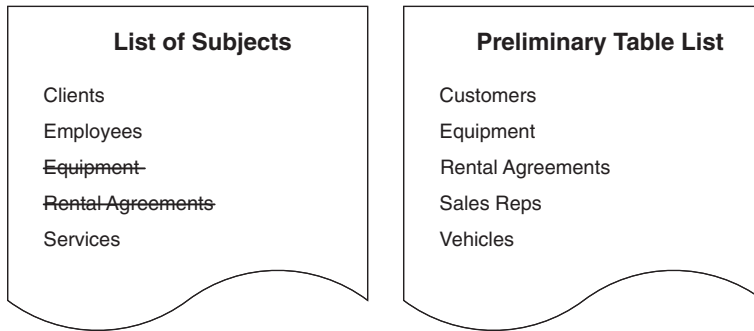


Figure 7.4 The revised list of subjects and the revised Preliminary Table List (first view).

Step 2: Resolve Items That Represent the Same Subject

Your objective during this step of the merge process is to determine whether an item on the list of subjects and an item on the Preliminary Table List *represent the same subject* even though they have *different* names. When you identify such a set of items, select the name that best represents the subject and use it as the sole identifier for that subject, and then deal with the name in this manner:

- If the name you’ve selected already appears on the Preliminary Table List, cross out its counterpart on the list of subjects.
- If the name appears on the list of subjects, *remove* its counterpart on the Preliminary Table List and *replace* it with the name from the list of subjects.

Repeat this process until you’ve covered all the items on the list of subjects.

Continuing with the equipment rental business example, assume you've discovered that "Clients" and "Employees" on the list of subjects and "Customers" and "Sales Reps" on the Preliminary Table List represent (respectively) the same subject (refer to Figure 7.4). Deciding to deal with "Clients" and "Customers" first, you review your interview notes and determine that "Customers" is the name that best represents both the people and the organizations that rent equipment from the business. You then resolve the duplication by keeping "Customers" and crossing out "Clients." Moving on to the next set of duplicate items, you decide to keep "Employees" and discard "Sales Reps" because you believe that "Employees" best describes those people who are employed by the business, regardless of their position. Figure 7.5 shows a revised version of both lists and the resolution of the duplicate items.

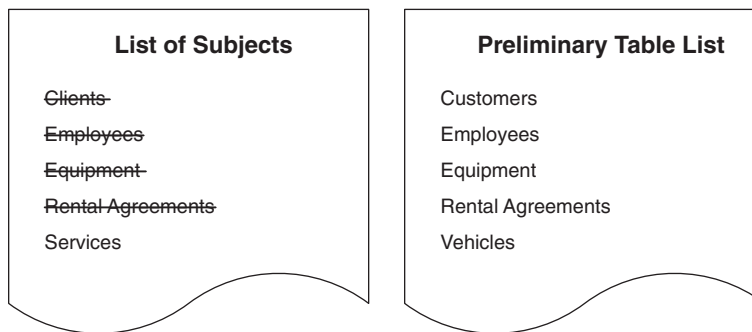


Figure 7.5 *The revised list of subjects and the revised Preliminary Table List (second view).*

Step 3: Combine the Items on the List of Subjects and the Preliminary Field List

The final step of this process is the easiest of the three. All you do is add the remaining items from the list of subjects to the Preliminary Table List. Throw away the list of subjects—you won't need it anymore. The list that remains becomes the second version of the Preliminary Table List. That's all there is to it! Figure 7.6 shows the second version of the Preliminary Table List, which is the result of merging the two lists shown in Figure 7.5.

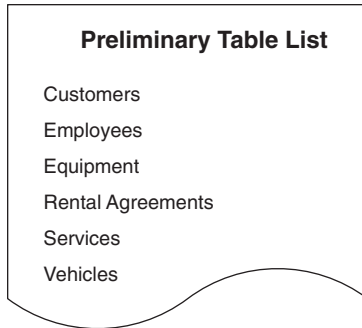


Figure 7.6 *The second version of the Preliminary Table List.*

Using the Mission Objectives

In this third and final procedure, you use the mission objectives to determine whether you've overlooked any subjects during the previous two procedures. This is your final opportunity to add tables to the Preliminary Table List.

Start with the first mission objective and use the Subject-Identification Technique to identify the subjects represented in that statement. Jot down each subject you identify on a separate piece of paper and then cross-check it against the items on the Preliminary Table List. Use the same techniques here that you used in the previous procedure.

1. When an item you jotted down from the mission objective statement matches an item on the Preliminary Table List, determine whether the items represent different subjects. If they do, assign an appropriate name to each occurrence and then add each one to the Preliminary Table List; otherwise, cross out the duplicate item that you jotted down.
2. When an item you jotted down from the mission objective statement has a name that is synonymous with the name of an item on the Preliminary Table List and both items represent the same subject, select the name that best identifies that subject and use it in the Preliminary Table List.

3. When an item you jotted down from the mission objective statement represents a new subject, add it to the Preliminary Table List.

Repeat these steps until you've worked through all the mission objectives. Here's an example of how you use these techniques to review the mission objectives.

Assume that you're designing a database for a flight training school. You're just starting this particular process, and you've just used the Subject-Identification Technique on the following statement:

We need to maintain data on our pilots and their certifications.

You now cross-check the subjects you identified in this mission objective against the items in the Preliminary Table List shown in Figure 7.7.

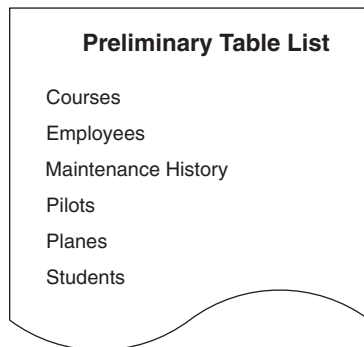


Figure 7.7 *The Preliminary Table List for a flight training school.*

In this case, you cross out “pilots” in the mission objective statement because it already exists on the Preliminary Table List, and it represents the same subject. You then decide to examine “certifications” further, and, after some careful thought, you make these observations.

- It does not appear on the Preliminary Table List.
- It doesn't duplicate any item on the Preliminary Table List.

- Its name is not synonymous with any item on the Preliminary Table List.
- It doesn't represent the same subject as any other item on the Preliminary Table List.

These findings indicate that “certifications” is a new item and should be added to the Preliminary Table List. So, you add it to the Preliminary Table List and cross it out on the mission objective statement; this shows you that you’ve already dealt with this particular item. Figure 7.8 shows the revised version of the Preliminary Table List.

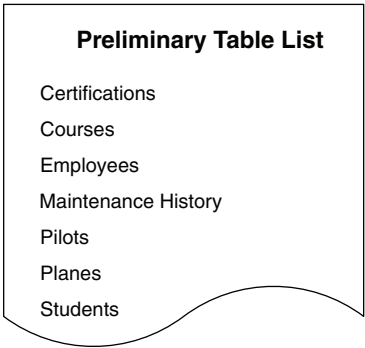


Figure 7.8 The revised Preliminary Table List.

Defining the Final Table List

Your Preliminary Table List is as complete as it can be at this point, so you'll now transform it into a *Final Table List*. This new list incorporates two elements that are not currently on the Preliminary Table List: *table type* and *table description*. Figure 7.9 shows an example of a Final Table List.

Final Table List		
Name	Type	Description
Classrooms	Data	The spaces or areas within a facility reserved for the purpose of conducting class proceedings. Information regarding the physical aspects, on-site resources, and availability of these areas is useful because it allows us to assign classes to the facility that can make the best use of these areas.
Courses	Data	The programs of instruction conducted through courses offered by this institution. Course information must always reflect the addition of new courses, the deletion of old courses, and the continuing evolution of existing courses.

Figure 7.9 An example of a Final Table List.

A *table type* allows you to classify a table by the role it plays within the database and provides you with a means of identifying tables that function in a similar manner. The table's role determines its type, and you can associate four table types with a given table:

1. A *data table* represents a subject that is important to the organization and is the primary foundation of the information that the database provides. (You'll learn more about data tables later in this chapter.)
2. A *linking table* establishes a link between two tables in a many-to-many relationship. (Chapter 10, "Table Relationships," covers linking tables in more detail.)
3. A *subset table* contains fields that are related to a particular data table and further describes the data table's subject in a very specific manner. (You'll learn more about subset tables later in this chapter.)

4. A *validation table* contains relatively static data and is a crucial component of data integrity. (Chapter 11, “Business Rules,” provides further details on this type of table.)

A *table description* provides a clear definition of the subject represented by the table and states why the subject is important to the organization. Certain guidelines govern how you create a table description, and you’ll learn about them later in this chapter. You have a final task to perform before you transform your Preliminary Table List into the Final Table List: refining the table names.

Refining the Table Names

Naming a table is a more complex affair than you may realize at the moment. As you learned in Chapter 3, “Terminology,” a table represents a single subject; therefore, its name must clearly identify the subject it represents.

Guidelines for Creating Table Names

The following guidelines will help you create table names that are clear, unambiguous, descriptive, and meaningful. They will also help ensure that you name your tables in a consistent manner.

- *Create a unique, descriptive name that is meaningful to the entire organization.* Using unique names helps to ensure that each table clearly represents a different subject and that everyone in the organization will understand what the table represents. (If you encounter duplicate table names at this point, resolve the problem using the techniques you learned earlier in this chapter.) Choose names that are descriptive enough to be self-explanatory. “Vehicle Maintenance” is an example of a good, descriptive name. Defining a unique and descriptive name does take some work on your part, but it’s well worth the effort in the long run.

- *Create a name that accurately, clearly, and unambiguously identifies the subject of the table.* Vague or ambiguous names usually indicate that the table represents more than one subject. When you encounter such a name, identify the subjects the table truly represents and then treat each subject as a separate table. “Dates” is a good example of a vague table name. You really don’t know what the table represents without referring to its description. For example, assume you’re designing a database for an entertainment agency and this table appears in the Preliminary Table List. Upon seeing this table name, you decide to review your interview notes. You discover that one person says “Dates” represents appointments for client meetings, and another person says it represents booking dates for the agency’s stable of entertainers. This table clearly represents two subjects, so you remove “Dates” from the Preliminary Table List and replace it with two new tables called “Client Meetings” and “Entertainer Schedules.”

Possibly the most vague and ambiguous name you could assign to a table is “**Miscellaneous.**” It doesn’t identify a single subject whatsoever. You might occasionally feel compelled to create a “Miscellaneous” table because you just can’t figure out what to do with certain fields on your Preliminary Field List. When that happens, stop, take a break, and then come back and re-examine those fields. Carefully and methodically apply the design techniques you’ve learned, and you’re sure to determine what to do with the fields after all.

- *Use the minimum number of words necessary to convey the subject of the table.* Everyone in the organization should be able to identify what the table represents without having to read its description. Although your objective is to create a short, succinct table name, avoid using a minimalist approach. “TD_1” is a good example of a name that is exceedingly short. You won’t have the slightest idea what this table represents unless you know the

meaning of each character in the name. You should also avoid going in the opposite direction as well. “Multiuse Vehicle Maintenance Equipment” is much too long and can easily be shortened to just “Equipment.”

- *Do not use words that convey physical characteristics.* Avoid using words such as *file*, *record*, and *table* in the table name because they add a level of confusion that you don’t need. A table name that includes this type of word is very likely to represent more than one subject. Consider the name “Patient Record.” On the surface, this may appear to be an acceptable name. You’ll realize, however, that there are potential problems with this name when you take some time to think about what a “patient record” is supposed to represent. The name contains a word that you’re trying hard to avoid (*record*), and it potentially represents three subjects: “patients,” “doctors,” and “examinations.” With this in mind, remove “patients” from the Preliminary Table List and replace it with three new tables, one for each of the three subjects.
- *Do not use acronyms and abbreviations.* Acronyms are hard to decipher, abbreviations rarely convey the subject of the table, and both violate the first guideline in this list. Take acronyms, for example. Say you’re helping an organization revise its database structure and you encounter a table named “SC.” How do you know what the table represents without knowing the meaning of the letters themselves? The fact is that you can’t easily identify the subject of the table. What’s more, you may find that the table means different things to different departments in the organization. So, you decide to conduct a brief interview with some of the staff to determine what the letters represent. (Now, this is the scary part.) To your disbelief, you discover that the folks in *personnel* think it stands for “Steering Committees”; the *information systems* staff believes it to be “System Configurations”; and the people in *security* insist that it represents “Security Codes.” This

example clearly illustrates why you should make every effort to avoid using abbreviations and acronyms in a table name.

- *Do not use proper names or other words that will unduly restrict the data that can be entered into the table.* This guideline will keep you from falling into the trap of creating duplicate table structures. A name such as “Southwest Region Employees,” for example, severely restricts the data that you can enter into this table. As the organization grows, how will you deal with employees from other regions? When the organization begins to hire employees in Washington, Oregon, and Idaho, you’ll have to create a “Pacific Northwest Region Employees” table, and you’ll have to create a “Western Region Employees” table when the organization begins to hire folks in Arizona, Utah, Nevada, and California.

Proper database design principles dictate that you should not create duplicate structures such as these because they can be quite problematic.

1. Users could have a difficult time retrieving data from all three tables simultaneously.
 2. The person maintaining the database would have the added responsibility of ensuring that the tables are always structurally synchronized. If he adds, modifies, or deletes a field in one table, he must take the same action on all the other tables.
 3. The person maintaining the database would also have the added responsibility of ensuring synchronized data integrity between the tables. He must be able to guarantee that data is completely and accurately transferred from one table to the other when an employee relocates from one region to another.
- *Do not use a name that implicitly or explicitly identifies more than one subject.* This is one of the most common mistakes you can

make with a table name, and it is relatively easy to identify. This type of name typically contains the word *and* or *or* and characters such as the slash (\) or ampersand (&); examples include “Department or Branch” and “Facility\Building.” A table with an ambiguous name suggests that you may not have identified the subject clearly or accurately during the analysis and interview processes. You can rectify this problem by reviewing your notes and conducting further analysis and interviews as necessary. Just remember that you must always ensure that each table represents only one subject.

Another name that falls under this category is “Miscellaneous.” (Yes, here’s that name again!) A moment ago, I said that this name didn’t identify a single subject whatsoever; this is a correct and valid assertion. It is also true, however, that the name implicitly identifies more than one subject; you can’t specifically identify the subjects because the name is vague and ambiguous. *Merriam-Webster’s online dictionary* defines the word itself as follows:

Miscellaneous *adj.* 1. consisting of diverse things or members; heterogeneous. 2. having various traits.

You can clearly see the problems that this name creates, so you should not use it as a table name at all. There are certainly good reasons not to do so.

- *Do use the plural form of the name.* As you know, a table represents a single subject, which can be an object or event. You can take this definition one step further and state that a table represents a *collection* of *similar* objects or events. For example, a sales representative wants to maintain data on *all* of his customers, not just a single one; and a car rental business wants to keep track of *all* its vehicles, not just the blue BMW. Using the plural form of the table name is a sound idea because it makes

clear your intention to refer to a collection. Collections, of course, always take the plural (“Boats,” not “Boat”). In contrast, words that identify fields are always singular (“Home Phone,” not “Home Phones”). *Following this rule will make it easy for you to differentiate between table names and field names in any documentation you create for the database.* (As you rename your tables, remember that the plural form of some words does not end in -s or -es. For instance, the singular and plural forms of “equipment” are exactly the same.)

Use these guidelines to refine each table name on the Preliminary Table List. When you’re finished, this list becomes your Final Table List and remains so for the duration of the database design process. Note that the list is “final” only in the sense that you’ve accounted for all the tables that you identified throughout the entire analysis process. It’s very likely that you’ll add new tables to this list based on requirements imposed by relationships, data integrity, or other information that you develop.

❖ **Note** The guideline for using a plural form for a table name is a particularly good one while you’re working on the logical design of the database. It makes it very easy to differentiate table names from field names, especially when you’re displaying them on a projection screen or when you’ve written them all across a white board in a conference room.

Keep in mind, however, that the table names are likely to change after you (or the database developer in charge of implementing the database) begin implementing the database into a specific RDBMS application. The names will then need to conform to the naming conventions that are standard for your organization or that developers commonly use for the RDBMS.

Indicating the Table Types

As you learned earlier in this chapter, you indicate each table's type on the Final Table List. Recall that the four classifications you can use to identify the table type are *data*, *linking*, *subset*, and *validation*.

When you first create your Final Table List, every item on the list is a *data* table because it represents a subject that is important to the organization and serves as the primary foundation of the information that the database provides. There will be no *linking* tables or *validation* tables on the list because you have not yet defined relationships or imposed data integrity. (You'll address these issues later in the design process.) The list will not contain *subset* tables because you define them *after* you assign fields to the data tables.

For the moment, designate each table on the Final Table List as a data table. You'll assign other table types later as the database design process continues to unfold.

Composing the Table Descriptions

The *table description* is another aspect of a table that you record on the Final Table List. A table description is crucial because it helps everyone understand why a given table exists and why the organization is concerned with collecting the data for that table. In fact, the description must *explicitly define the table* and *state its importance* to the organization. It doesn't matter whether the definition comes first, or you use more than one sentence to convey this information. Both the definition and the explanation of the table's importance must be in the description. The table description also provides a means of validating the need for a table. If you are unable to explain why a table is important to the organization, then you need to determine when and how the table was identified and whether it is necessary at all.

Guidelines for Composing a Table Description

Just as you had guidelines to help define table names, you also have a set of guidelines to help you compose a focused, concise, unambiguous, and clear table description:

- *Include a statement that accurately defines the table.* Anyone should easily be able to determine the identity of the table from its description without any confusion or uncertainty. Here's an example of a poor definition for a table named "Suppliers" in a bakery database. As you can see, it's not very accurate:

Suppliers—The companies that supply us with ingredients and equipment

What if the bakery receives some of its ingredients from local small farmers? Small farmers certainly don't qualify as "companies." What type of equipment do these suppliers supply? Cooking utensils? Hand trucks? Delivery racks? Here's a much better definition of suppliers:

Suppliers—The people and organizations from which we purchase ingredients and equipment

This statement can be used in the table description as the *table definition*.

- *Include a statement that explains why this table is important to the organization.* A table contains data that is collected, maintained, manipulated, and retrieved by the organization for a *particular* reason. Your statement should explain *why* the data is important to the organization. Keeping in mind that this statement becomes part of your table description, you might be tempted to construct a statement such as this:

We need the Suppliers table to keep track of the names, addresses, phone numbers, and contact names of all our suppliers.

This statement is inadequate because it emphasizes only *what* needs to be stored in the Suppliers table instead of amplifying *why* the data is important to the business. The next example conveys a better sense of why the information is important:

Supplier information is vital to the bakery because it allows us to maintain a constant supply of ingredients and ensure that our equipment is always in working order.

This is a more effective statement because it conveys the importance of the data by identifying the services the suppliers provide to the bakery. It also implies that the bakery could run out of ingredients or have a hard time keeping its equipment in top shape without the suppliers' services. This statement now reflects why the table is important to the organization.

- *Compose a description that is clear and succinct.* Avoid the common mistake of restating or rephrasing the table name in your table description, as in this example:

Student Schedule—The class schedule of the student

Don't be too brief or too verbose. You want to make sure that everyone can identify the table and understand its importance to the organization, but you also want to avoid furnishing too much information. Here's an example of a description that is quite lengthy and provides more information than is necessary:

Student Schedule—All the classes that a student will attend (including the days, the times, and the faculty conducting the class) during the course of the school year. The data in this table is important because it will let the student know the name of the class and when and where he's supposed to be. Also, the student will know the duration of the class, as well as the name of the teacher who is teaching the class.

This can be recast more clearly and succinctly as follows:

Student Schedule—Those classes that the student is scheduled to attend during this school year. The information provided by this table helps the student implement effective time management and enables the school to figure class loads and student loads.

The first sentence in this example provides the definition of the table, and the second sentence states why the table is important to the academic organization.

- *Do not include implementation-specific information in your table description, such as how or where the table is used.* Avoid statements that indicate how you will specifically use this table, or how you will physically access it. This type of information is germane to the database *implementation* process, which is wholly separate from the *database design* process you're learning in this book. Here is an example of a description containing this type of inappropriate information:

Student Schedule—Those classes that the student is scheduled to attend during this school year. This information is used by the registrar and is accessed from the Student Admissions menu in the Registration Program.

- *Do not make the table description for one table dependent upon the table description of another table.* Each table description should be self-explanatory and independent from every other table description; it should be absolutely unnecessary for you to cross-reference one table description against another. This is the type of statement you're trying to avoid:

Dependents—The spouse, children, or wards of a given employee. (See description of Employee table for further information.)

Here's a much better description:

Dependents—The spouse, children, or wards of a given employee. This information allows us to make the appropriate tax deductions for the employee and is necessary for the benefits programs in which the employee is enrolled.

- *Do not use examples in a table description.* An example is a valuable communication tool that helps you convey a particular meaning or concept and is very effective when you use it wisely. However, an example depends on supplemental information (and, in some cases, further examples) to complete the idea it is supposed to convey. For instance, just think of the number of examples you would have to use to define fully what a table represents. A well-defined description is clear, succinct, and self-explanatory; therefore, it does not require an example to convey its meaning.

Interviewing Users and Management

Now you'll define table descriptions for the tables on the Final Table List. You'll conduct interviews with both users and management and enlist their aid in establishing each table's definition and importance to the organization. (This is one of the few times that you'll actually interview both groups together.) Your main objective is to get a consensus on general descriptions for the tables. When your interviews are complete, take your notes and compose final table descriptions, making sure to follow the guidelines outlined earlier in this chapter. Then confer with both parties once more to make certain that the descriptions are acceptable and easily understood by all. The Final Table List is complete when everyone has agreed on the descriptions.

Consider this example: Assume you're developing a database for a local software training organization. Your assistant, John, is conducting an interview with some of the people from the organization. Specifically, he's speaking to Mark from the administration department; Jason, the instructor coordinator; Sara, the vice president of sales; and Carol, the

head of the organization. The dialogue that follows is a partial transcript of John's interview. John is currently discussing the Students table.

❖ **Note** Unlike the interviews you conducted during the analysis and requirements review stages of the design process, you no longer need to involve everyone in the organization. However, you will work with a representative group of users and management for the interviews you'll conduct throughout the remainder of the design process.

JOHN: "Okay, let's talk about the Students table. How would you describe a 'student'?"

JASON: "A student is a private individual who comes in for one of our classes."

SARA: "That's only partially true. A student can also be an individual that an organization sends to our classes. For example, many of our students come from local banks and insurance companies, and those organizations pay for the students' tuitions."

MARK: "Makes sense. I guess we can simply say that a student is an individual who comes in for one of our classes."

(John makes a note of what Mark just said.)

JOHN: "Good—got it. Does anyone have anything else to add?"
(Pause...)

"Great. Now, how would you explain to someone why student information is important to this organization?"

CAROL: "Without students, we don't have a business!"

JASON: "If we can keep track of the students who attend our classes, we can send them information regarding our new classes."

SARA: “Keeping track of this information allows us to keep billing and contact information current. This is especially true for organizations that send their employees to our classes. Training coordinators move on to other positions, and we have to know the name of the new person we’ll be dealing with.”

JOHN: “Good point. Does anyone have anything further to add? Okay, does everyone agree with what has been said so far?”

(Everyone nods in approval. Because no additional comments are made, John jots down some final notes and moves on to the next table.)

As you can see, conducting this type of interview is a fairly straightforward affair. Notice how John attempts to get a consensus as he recognizes that no one has anything else to say about the topic at hand. He then makes note of the points that will help him compose the description and moves on to his next topic.

After John has finished conducting the interview, he uses his notes to develop a table description for each table on the Final Table List. He’ll have to interpret and study the participants’ responses to develop a suitable table description. Based on his examination, John writes the following description:

Students—Those individuals who attend our classes. The information provided by the data in the Students table allows our organization to further promote our classes and supports proper communications with the students.

John then writes a description for each table on the Final Table List. When he’s finished, he’ll speak with Mark, Jason, Sara, and Caroline once more to make sure the descriptions are acceptable and that everyone understands them without any difficulty.

Associating Fields with Each Table

In Chapter 3 you learned that tables are composed of fields. During this stage of the database design process, you'll assign fields to each table on the Final Table List using fields from your *Preliminary Field List*.

Assigning fields to a table is a relatively easy process: Determine which fields best represent characteristics of the table's subject and assign them to that table. Repeat this procedure for every table on the Final Table List. If you think you can use a field or set of fields to represent characteristics of more than one table, then assign them accordingly. You'll discover whether you've assigned the appropriate fields to each table later when you go through the process of refining the table structures.

❖ **Note** In the following examples, you'll note that I ask you to use sheets of paper for specific procedures. Using paper helps you avoid the temptation of using an RDBMS program to design your database. I cannot overemphasize or overstate the fact that you should not use the computer at all until the database design process is complete *unless* you're using some type of *database design-specific* software. By heeding this advice, you will avoid the traps I discuss later in Chapter 14, "Bad Design—What Not to Do."

Begin this process by taking a sheet of paper and laying it in front of you lengthwise from left to right. Write the name of each table (from the Final Table List) across the top of the paper, starting at the left-hand side; leave enough space between the table names to give you enough room to list lengthy field names underneath them. Repeat this procedure, using as many sheets as you need to account for every table on the list. Continuing with the school database example, Figure 7.10 shows the set of table structures currently under development.

Table Structures			
Courses	Subjects	Instructors	Students

Figure 7.10 Setting up a sheet for listing table structures.

Next, assign fields from the Preliminary Field List to each table. Determine which fields best describe or define a table’s subject and then list these fields underneath the table name. After you’ve assigned all the fields you believe to be appropriate for the table, move on to the next table and repeat the process. Continue in this manner until you’ve assigned fields to all the tables. Figure 7.11 shows a partial set of table structures.

Table Structures			
Classes	Subjects	Instructors	Students
Class Number	Subject Name	Instructor Name	Student Name
Class Name	Subject Description	Instructor SSN	Student SSN
Subject Name	Category	Instructor Address	Student Address
Instructor Name	Credits	Instructor Phone	Student Phone
Room Number		Date Hired	
		Pay Rate	

Figure 7.11 Listing tables with their associated fields.

❖ **Note** Before you work through the remainder of the chapter, now is a good time to recall a principle I presented in the Introduction:

Focus on the concept or technique and its intended results, not on the example used to illustrate it.

I bring this to your attention once again because you'll certainly wonder why I created an example in a particular manner. Maybe you've thought of a different or better approach to the problem, and you might have thoroughly valid reasons for using it. Don't let the example mislead you. I've fashioned each example in a specific manner for the sole reason of illustrating the concept or technique at hand. Therefore, study the way that I correct the problems you see in a particular example so that you can use those techniques when you encounter similar problems in your database.

Refining the Fields

Now that you've assigned fields to each table, you'll refine the fields by improving the field names and resolving any structural problems that may exist. You'll then refine the tables further by establishing that you've assigned the appropriate fields to each table and that the table structures are sound.

Improving the Field Names

As you know, a field represents a characteristic of the subject of the table to which it belongs. You can easily identify the characteristic a field is supposed to represent when that field has an appropriate name. A field name that is ambiguous, vague, or unclear is a sure sign of trouble and suggests that you have not thoroughly identified the purpose of the field.

Guidelines for Creating Field Names

Earlier in this chapter, you learned a set of guidelines for naming a table. Now you'll learn another set of guidelines that you'll apply to field names. Fortunately, many of them are similar to the guidelines governing table names, so you're already familiar with most of the concepts:

- *Create a unique, descriptive name that is meaningful to the entire organization.* A given field name should appear only once in the entire database; the only exception to this rule occurs when the field serves to establish a relationship between two tables. Make certain the name is descriptive enough to convey its meaning accurately to everyone who sees it. (Chapter 10 covers this issue in greater detail.)
- *Create a name that accurately, clearly, and unambiguously identifies the characteristic a field represents.* “Phone Number” is a good example of an inaccurate, ambiguous field name. What kind of phone number does it represent? A home phone? An office phone? A cell phone? Learn to be specific. If you need to record each of these types of phone numbers, then create “Home Phone,” “Work Phone,” and “Cell Phone” fields.

In Chapter 6, “Analyzing the Current Database,” you learned how to resolve generic field names, such as “Address,” “City,” and “State,” by using the *table* name as a prefix for the field name. This produces names such as “*Employee* Address,” “*Customer* Address,” and “*Supplier* Address.” When you have field names such as these, you can abbreviate the prefix (for brevity's sake) by using the first three or four letters of the table name as the revised prefix. This allows you to transform the previous field names into “*Emp*Address,” “*Cust*Address,” and “*Supp*Address.” This technique helps you fulfill not only this guideline, but the previous one as well.

❖ **Note** The degree to which you use prefixes within a table is a matter of style. When a table contains generic field names, some database designers will choose to prefix the generic names only, whereas others elect to prefix *all* the field names within the table. Regardless of the prefix method you choose to use, using it consistently throughout the database structure is very important.

I personally prefer to prefix the generic field names only, and I'll follow this preference throughout the remainder of the book.

- *Use the minimum number of words necessary to convey the meaning of the characteristic the field represents.* You want to avoid lengthy field names, but at the same time, you also want to avoid using a single word as a field name if that word is inappropriate. For example, if you're trying to record the date a particular employee joined the organization, "Hired" is too short (and slightly vague) and "Date That the Employee Was Hired" is too long! "Date Hired," however, is a more appropriate name and accurately represents the characteristic the field represents.
- *Do not use acronyms and use abbreviations judiciously.* Acronyms can be hard to decipher and often lead to misunderstanding. Imagine a field named "CAD_SW." How would you determine what the field represents? On the other hand, you can use abbreviations as long as you use them sparingly and handle them with care. Only use an abbreviation if it supplements or enhances the field name in a positive manner. An abbreviation shouldn't make a field name ambiguous or diminish its meaning.
- *Do not use words that could confuse the meaning of the field name.* A field name that contains redundant words or synonyms can make the name's meaning unclear and subject to misinterpretation. For instance, consider the name "Digital Identification Code Number." "Digital" and "number" are redundant, so you can eliminate either one without diminishing the field name's meaning.

Let's assume that you decide to eliminate "digital." You can split the remaining name into two smaller names: "Identification Code" and "Identification Number." These names are often synonymous, and you can easily use either as the final field name. In this situation, just use the name that is most meaningful within the organization.

- *Do not use names that implicitly or explicitly identify more than one characteristic.* These types of names are easy to spot because they typically use the word *and* or *or*. Field names that contain a slash (\) or an ampersand (&) are dead giveaways as well. When you encounter a field with a name such as "Area or Location" or "Phone\Fax," identify each characteristic that the name implies, and create a new field for the characteristic. Then test the new field name against these guidelines to ensure that the name is sound.
- *Use the singular form of the name.* A field with a plural name, such as "Skills," implies that it may contain two or more values for a given record, which is not a good idea. (You'll learn more about this later in the chapter.) A field name is singular because it represents a *single* characteristic of the subject of the table to which it belongs. A table name, on the other hand, is plural because it represents a *collection* of similar objects or events. You can distinguish table names from field names quite easily when you use this naming convention.

❖ **Note** The specific guideline for using a table name as a prefix for a field name will have the same issue that I brought up earlier for table names: These particular field names are likely to change once you (or the database developer in charge of implementing the database) begin implementing the database into a specific RDBMS application. The names will need to conform to the naming conventions that are standard for your organization or that developers commonly use for the RDBMS.

With these guidelines in mind, review each table and determine whether you can make improvements to any of the field names. When you're finished, you're ready to identify and resolve any problems with the fields. Figure 7.12 shows revisions to the field names of the table structures in Figure 7.11.

In Figure 7.12, “Classes” is shortened to “Cls,” “Subjects” is shortened to “Subj,” “Instructors” is shortened to “Inst,” “Student” is shortened to “Std,” and “Social Security Number” replaces “SSN.” Remember that abbreviations can be very useful as long as they are meaningful and understood by everyone in the organization. Using proper and appropriate abbreviations will not detract from the meaning of the field name.

Table Structures			
Classes	Subjects	Instructors	Students
ClsNumber	SubjName	InstName	StdName
ClsName	SubjDescription	InstSocial Security Number	StdSocial Security Number
SubjName	Category	InstAddress	StdAddress
InstName	Credits	InstPhone	StdPhone
Room Number		Date Hired	
		Pay Rate	

Figure 7.12 Revised field names.

❖ **Note** Throughout the remainder of the chapter and the rest of the book, table names within the text appear in all capital letters (such as VENDORS) and field names within the text appear in small capital letters (such as VENDOR ID NUMBER).

Using an Ideal Field to Resolve Anomalies

Although you've carefully identified the fields on your Preliminary Field List, you may have created a few fields that could prove problematic to the table structure. Poorly defined fields can cause duplicate data and redundant data, and they can be difficult to use. You might find it difficult to determine whether any of the fields in a table is going to cause problems unless you know the warning signs. The best way to identify potentially troublesome fields is to determine whether they comply with the Elements of the Ideal Field, as discussed next.

Elements of the Ideal Field

The Elements of the Ideal Field constitute a set of guidelines you can use to create sound field structures and to spot poorly designed fields easily. An ideal field has the following characteristics:

- *It represents a distinct characteristic of the subject of the table.* As you know, a table represents a specific subject, which can be an object or event. The ideal field represents a distinct characteristic of that object or event.
- *It contains only a single value.* A field that can potentially store two or more occurrences of the *same* value is known as a *multivalued* field. A multivalued field causes data redundancy problems (quite obviously) and is difficult to use when you try to edit, delete, or sort the data within it. The ideal field is free of these problems because it contains only a single value.
- *It cannot be deconstructed into smaller components.* A field that can potentially store two or more *distinct* items within a value is known as a *multipart* (or composite) field. Like the multivalued field, this type of field causes problems when you try to edit, delete, or sort the data within it. These problems don't occur with an ideal field because it represents a single, distinct characteristic of the subject of the table to which it belongs. (You'll learn more about multivalued and multipart fields in just a moment.)

- *It does not contain a calculated or concatenated value.* The values of the fields in a table should be mutually independent; a particular field should not have to depend on the values of other fields for its own value. A calculated field, however, *does* depend on the values of other fields for its own value, and therein lies the problem. The calculated field's value *is not updated* when the value of any field participating in the calculation changes. It then becomes the responsibility (and an undesirable burden) of the user or the database application program to update the calculated field when this type of change takes place. This is precisely why you deal with calculated fields separately.
- *It is unique within the entire database structure.* The only duplicate fields that appear in a properly designed database are those that establish relationships between tables. If duplicate fields other than these exist in a table, it is very likely that the table will accumulate unnecessary redundant data and that the data within the duplicate fields will inevitably become inconsistent.

❖ **Note** Remember that you're dealing strictly with the *logical* database structure at this point. You might have cause to duplicate specific fields when you *physically* implement the database in an RDBMS program. During that process, however, you're making a conscious decision to duplicate the fields and you're prepared to deal with the consequences of that decision.

- *It retains a majority of its properties when it appears in more than one table.* A field that establishes a relationship between two tables is a structural component of each table. A majority of the field's properties remain constant in each occurrence of the field. (Chapter 9, "Field Specifications," and Chapter 10 cover this matter in greater detail.)

Although you now know the specific elements of an ideal field, you'll still find it difficult in many instances to identify problematic fields just by looking at their names. Figure 7.13 shows a table structure that helps to illustrate this point. Take a moment and try to determine whether each field complies with the Elements of the Ideal Field or needs to be modified.

Table Structures	
Instructors	
InstName	
InstAddress	
InstPhone	
Categories Taught	
InstSocial Security Number	
Date Hired	
Pay Rate	

Figure 7.13 A table containing fields with questionable structures.

Each field on the list seems to conform to the Elements of the Ideal Field. Examine the list carefully, however, and you'll see that some fields don't really comply with the second and third elements. Three fields have anomalies that will cause problems unless you resolve them: INSTNAME, INSTADDRESS, and CATEGORIES TAUGHT. If you doubt this assertion, you can test it by "loading" the table with sample data. This will quickly reveal anomalies, if any exist, and is the best way to confirm whether a field complies with all the Elements of the Ideal Field.

You don't have to create a table physically to perform this test. Take a sheet of paper and lay it in front of you lengthwise from left to right. Write the name of each field across the top of the paper, starting from the left-hand side; leave enough space between the field names to allow room for the values you're going to place underneath them.

Enter records into the table by filling in each field with some sample data; be sure the sample data represents the data you're actually going to enter into the database. You need only a few records for the test to work properly. Your sheet of paper should look similar to the one in Figure 7.14.

Instructors

InstName	InstAddress	InstPhone	Categories Taught	<< other fields >>
Kira Bently	3131 Mockingbird Lane, Seattle, WA 98157	221 363-9948	DTP, SS, WP	
Bill Champlin	7402 Kingman Drive, Redmond, WA 98115	230 527-4992	WP, DB, OS	
Shannon Black	4141 Lake City Way, Seattle, WA 98136	230 336-5992	DB, SS	
Estela Rosales	970 Phoenix Avenue, Bellevue, WA 98046	221 336-6992	DTP, WP, PG	

Figure 7.14 Testing a table on paper with sample data.

❖ **Note** As I mentioned in Chapter 3, I show only those fields that are most relevant to the discussion at hand and use <<other fields>> to represent fields that are nonessential to the example.

Now you can easily identify which fields are going to be troublesome unless they are resolved. As you can see, INSTNAME and INSTADDRESS are both multipart fields, and CATEGORIES TAUGHT is a multivalued field. You must resolve these fields before you can refine the table structure.

Resolving Multipart Fields

Working with a multipart field is difficult because its value contains two or more *distinct* items. It's hard to retrieve information from a multipart field, and it's hard to sort or group the records in the table by the field's value. The INSTADDRESS field in Figure 7.14 illustrates these difficulties; you would certainly have problems retrieving information for the city of Seattle or sorting information by ZIP code.

You resolve a multipart field by identifying the distinct items within the field's value and treating each item as an individual field.

Accomplish this task by asking yourself a simple question: “What specific items does this field’s value represent?” After you’ve answered the question and identified the items (as best you can), transform each item into a new field.

In Figure 7.14, the value of the field `INSTNAME` represents two items: the first name and the last name of an instructor. You resolve this field by creating a new `INSTFIRST NAME` field and a new `INSTLAST NAME` field. The value of `INSTADDRESS` represents four items: the street address, city, state, and ZIP code of an instructor. You transform these items into fields as well; they will appear in the table as `INSTSTREET ADDRESS`, `INSTCITY`, `INSTSTATE`, and `INSTZIPCODE`. Figure 7.15 shows the newly revised `INSTRUCTORS` table.

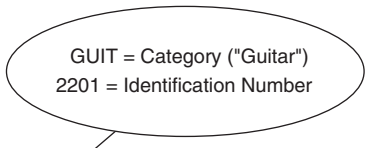
Instructors

InstFirst Name	InstLast Name	InstStreet Address	InstCity	InstState	InstZipcode	InstPhone	Categories Taught	<< other fields >>
Kira	Bently	3131 Mockingbird Lane	Seattle	WA	98157	221 363-9948	DTP, SS, WP	
Bill	Champlin	7402 Kingman Drive	Redmond	WA	98115	230 527-4992	WP, DB, OS	
Shannon	Black	4141 Lake City Way	Seattle	WA	98136	230 336-5992	DB, SS	
Estela	Rosales	970 Phoenix Avenue	Bellevue	WA	98046	221 336-6992	DTP, WP, PG	

Figure 7.15 Resolving the multipart fields in the `INSTRUCTORS` table.

Some multipart fields are hard to recognize. Take a look at the `INSTRUMENTS` table in Figure 7.16. At first glance, the table doesn’t seem to contain multipart fields. When you examine the data in the table more closely, however, you’ll see that `INSTRUMENT ID` is actually a multipart field. This field’s value represents two distinct items: the category to which the instrument belongs—AMP (amplifier), GUIT (guitar), MFX (multi-effects unit), SFX (single-effect unit)—and the instrument’s identification number. Clearly, you should deconstruct `INSTRUMENT ID` into two smaller fields in accordance with the third element of an ideal field. Imagine how difficult it would be for you to update the field’s value if the MFX category changed to MFU if you don’t do this. You would have to write programming code to parse the value, test for the existence of MFX, and then replace it with MFU if it existed within the

parsed value. It's not so much that you *can't* do this, but you would definitely be working harder than necessary, and you shouldn't have to go through this at all if you have a properly designed database.



Instruments

Instrument ID	Manufacturer	Instrument Description	<< other fields >>
GUIT2201	Fender	Stratocaster	
MF33349	Zoom	Player 2100 Multi-Effects	
AMP1001	Marshall	JCM 2000 Tube Super Lead	
AMP5590	Crate	VC60 Pro Tube Amp	
SFX2227	Dunlop	Cry Baby Wah-Wah	
AMP2766	Fender	Twin Reverb Reissue	

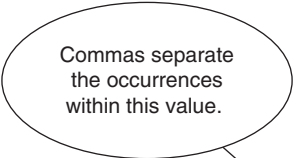
Figure 7.16 An example of a “hidden” multipart field.

Resolving Multivalued Fields

As you know, a multivalued field can potentially store two or more occurrences of the *same* value. Fortunately, you'll recognize a multivalued field when you see one. The field's name is often plural, and its value almost invariably contains a number of commas, which serve to separate the various occurrences that exist within the value itself.

Resolving multipart fields is not very hard at all, but resolving multivalued fields can be a little more difficult and will take some work. A multivalued field has the same fundamental set of problems as a multipart field, as the CATEGORIES TAUGHT field in Figure 7.17 clearly illustrates. For example, you'll have difficulty retrieving information for everyone who teaches a specific category (such as WP), you can't sort the data in any meaningful fashion, and, most importantly, you don't

have room to enter more than four categories. What happens when one or more instructors teach *five* categories? The only option you’ll have is to make the field larger every time you need to enter more values than it will currently allow.



Instructors

InstFirst Name	InstLast Name	InstStreet Address	InstCity	<< other fields >>	Categories Taught
Kira	Bently	3131 Mockingbird Lane	Seattle		DTP, SS, WP
Bill	Champlin	7402 Kingman Drive	Redmond		WP, DB, OS
Shannon	Black	4141 Lake City Way	Seattle		DB, SS
Estela	Rosales	970 Phoenix Avenue	Bellevue		DTP, WP, PG

Figure 7.17 Identifying a multivalued field.

So how would you resolve this multivalued field? Your first thought may be to create a new field for each value, thus “flattening” the multivalued field into several single-valued fields. Figure 7.18 shows what will happen if you follow through with this idea.

Instructors

InstFirst Name	InstLast Name	InstStreet Address	InstCity	<< other fields >>	Category Taught 1	Category Taught 2	Category Taught 3
Kira	Bently	3131 Mockingbird Lane	Seattle		DTP	SS	WP
Bill	Champlin	7402 Kingman Drive	Redmond		WP	DB	OS
Shannon	Black	4141 Lake City Way	Seattle		DB	SS	
Estela	Rosales	970 Phoenix Avenue	Bellevue		DTP	WP	PG

Figure 7.18 The result of “flattening” the CATEGORIES TAUGHT field.

Unfortunately, this is not much of an improvement at all. Three specific problems arise from this type of structure:

1. *Retrieving category information will be tedious at best.* A user attempting to find all instructors who teach the WP category must be sure to search for this value within each of the category

fields; there is no guarantee that WP is consistently stored in the same field. Failure to do so means that the user runs the risk of overlooking a qualified instructor.

2. There is no way for the RDBMS program to sort the category data in a meaningful fashion.
3. *This structure is inherently volatile.* In its current state, the table unnecessarily restricts the number of categories an instructor can teach; you must create additional category fields when you have instructors who teach more than three categories. Adding more category fields just compounds the first two problems.

Realizing that flattening the CATEGORIES TAUGHT field won't solve your problem, your next thought is to bring the field into compliance with the second element of an ideal field and declare that it will contain only a single value. Although this is a good impulse and a step in the right direction, it will not resolve the matter completely because it will introduce yet another problem: data redundancy. Figure 7.19 illustrates what happens when you follow through with this particular idea. Note that there is now a single value in the CATEGORIES TAUGHT field for each record in the table.

Instructors

InstFirst Name	InstLast Name	InstStreet Address	InstCity	InstState	InstZipcode	InstPhone	Categories Taught
Kira	Bently	3131 Mockingbird Lane	Seattle	WA	98157	221 363-9948	DTP
Kira	Bently	3131 Mockingbird Lane	Seattle	WA	98157	221 363-9948	SS
Kira	Bently	3131 Mockingbird Lane	Seattle	WA	98157	221 363-9948	WP
Bill	Champlin	7402 Kingman Drive	Redmond	WA	98115	230 527-4992	WP
Bill	Champlin	7402 Kingman Drive	Redmond	WA	98115	230 527-4992	DB
Bill	Champlin	7402 Kingman Drive	Redmond	WA	98115	230 527-4992	OS
Shannon	Black	4141 Lake City Way	Seattle	WA	98136	230 336-5992	DB
Shannon	Black	4141 Lake City Way	Seattle	WA	98136	230 336-5992	SS
Estela	Rosales	970 Phoenix Avenue	Bellevue	WA	98046	221 336-6992	DTP
Estela	Rosales	970 Phoenix Avenue	Bellevue	WA	98046	221 336-6992	WP
Estela	Rosales	970 Phoenix Avenue	Bellevue	WA	98046	221 336-6992	PG

Figure 7.19 The result of bringing CATEGORIES TAUGHT into compliance with the second element of an ideal field.

The values in CATEGORIES TAUGHT cause redundant data because you must duplicate a given instructor record *for each category* that the instructor teaches. This redundancy is obviously unacceptable, so you'll have to resolve this problem in some other manner.

You can avoid this situation entirely by using these steps to resolve a multivalued field:

1. Remove the field from the table and use it as the basis for a new table. If necessary, rename the field in accordance with the field name guidelines that you learned earlier in this chapter.
2. Use a field (or set of fields) from the *original* table to relate the original table to the new table; try to select fields that represent the subject of the table as closely as possible. The field(s) you choose will appear in both tables. (You'll learn more about relating tables in Chapter 10.)
3. Assign an appropriate name, type, and description to the new table and add it to the Final Table List.

These steps form a generic procedure that you can use to resolve any multivalued field you encounter in a table. Now, apply these steps to the CATEGORIES TAUGHT field.

1. Remove the field from the INSTRUCTORS table and use it as the basis of a new table. Because this will now be a *single*-valued field, rename the field CATEGORY TAUGHT.
2. Use INSTFIRST NAME and INSTLAST NAME as the connecting fields that will relate the INSTRUCTORS table to the new table and add them to the structure of the new table.
3. Give the new table a proper name, compose a suitable description, and add the table to the Final Table List. (Indicate the

table's type as "Data.") Here are one possible name and description you might use for the new table:

Instructor Categories—The categories of software programs that an instructor is qualified to teach. The information this table provides allows us to make certain that an adequate number of instructors exists for each software category.

Figure 7.20 shows the revised INSTRUCTORS table and the new INSTRUCTOR CATEGORIES table.

Instructors

InstFirst Name	InstLast Name	InstStreet Address	InstCity	InstState	InstZipcode	InstPhone
Kira	Bently	3131 Mockingbird Lane	Seattle	WA	98157	221 363-9948
Bill	Champlin	7402 Kingman Drive	Redmond	WA	98115	230 527-4992
Shannon	Black	4141 Lake City Way	Seattle	WA	98136	230 336-5992
Estela	Rosales	970 Phoenix Avenue	Bellevue	WA	98046	221 336-6992

Instructor Categories

InstFirst Name	InstLast Name	Category Taught
Kira	Bently	DTP
Kira	Bently	SS
Kira	Bently	WP
Bill	Champlin	WP
Bill	Champlin	DB
Bill	Champlin	OS
Shannon	Black	DB
Shannon	Black	SS

Figure 7.20 Resolving the multivalued field in the INSTRUCTORS table.

Note that the new INSTRUCTOR CATEGORIES table is free from the problems typically associated with multivalued fields because CATEGORY TAUGHT is a *single-value* field. You can easily retrieve information for a particular instructor or category, and you can sort the records in a meaningful manner. Also note that the INSTFIRST NAME and INSTLAST

NAME fields retain their names in the new table, making them compliant with the fifth element of an ideal field.

Although the new table contains redundant data, the redundancy is acceptable because it is *minimal*. It's a fact of life that a relational database will always contain some amount of redundant data. Your goal as the database architect is to make certain that it has only an *absolute minimum* amount of redundant data.

Figure 7.21 shows a version of the INSTRUCTORS table that contains *three* multivalued fields.

- CATEGORIES TAUGHT—This indicates the categories of classes that an instructor can teach.
- MAXIMUM LEVEL TAUGHT—This indicates the maximum skill level that the instructor can teach for a given category.
- LANGUAGES SPOKEN—This indicates the foreign languages that an instructor can speak.

Instructors

InstFirst Name	InstLast Name	Campus Phone	Categories Taught	Maximum Level Taught	Languages Spoken
Kira	Bently	221 363-9948	DTP, OS, SS, WP	Intermediate, Basic, Advanced, Basic	French, Spanish
Bill	Champlin	230 527-4992	DB, OS, UT, WP	Intermediate, Basic, Basic, Advanced	German, Spanish
Shannon	Black	230 336-5992	DB, PG, SS	Advanced, Intermediate, Intermediate	French, German
Estela	Rosales	221 336-6992	DTP, PG, WP	Basic, Intermediate, Basic	French, Italian, Spanish

Figure 7.21 A version of the INSTRUCTORS table containing three multivalued fields.

Your task here seems relatively clear—you're going to use the procedure you've just learned to resolve these multivalued fields. You then notice one small, relatively obscure problem: There is a distinct one-to-one association between values in CATEGORIES TAUGHT and the values in MAXIMUM LEVEL TAUGHT for any given record. You probably wouldn't have noticed this anomaly had you not carefully examined the sample data

within these fields. Don't worry; you'll still use the same procedure, but with one minor modification.

You'll occasionally encounter a situation such as this, where some given field (whether single- or multivalued) depends on a particular multivalued field. You can easily fix this problem by including the dependent field in the structure of the new table you build to resolve the multivalued field. Figure 7.22 shows the results of consolidating this technique with the previous one to resolve CATEGORIES TAUGHT. (It shows the resolution of LANGUAGES SPOKEN as well.)

The redundancy in the new tables is acceptable because, once again, it is minimal. In Chapter 10, you'll learn how to reduce this type of redundancy even further by relating the tables with primary keys and foreign keys.

Instructors

InstFirst Name	InstLast Name	Campus Phone
Kira	Bently	221 363-9948
Bill	Champlin	230 527-4992
Shannon	Black	230 336-5992
Estela	Rosales	221 336-6992

Instructor Categories

InstFirst Name	InstLast Name	Category Taught	Maximum Level
Kira	Bently	DTP	Intermediate
Kira	Bently	OS	Advanced
Kira	Bently	SS	Basic
Kira	Bently	WP	Advanced
Bill	Champlin	DB	Intermediate
Bill	Champlin	OS	Basic
Bill	Champlin	UT	Basic
Bill	Champlin	WP	Advanced

Instructor Languages

InstFirst Name	InstLast Name	Language Spoken
Kira	Bently	French
Kira	Bently	Spanish
Bill	Champlin	German
Bill	Champlin	Spanish
Shannon	Black	French
Shannon	Black	German
Estela	Rosales	French
Estela	Rosales	Italian
Estela	Rosales	Spanish

Figure 7.22 Resolving the multipart fields in the INSTRUCTORS table.

Refining the Table Structures

Now that you've refined the fields and made certain that each field is sound, you can begin the process of refining the table structures. Your objective in this phase of the design process is to make sure that you've assigned the appropriate fields to each table and that you've properly defined each table's structure. This process will also reveal whether the tables have anomalies that you need to resolve.

A Word about Redundant Data and Duplicate Fields

You've seen the term *redundant data* used quite often in this chapter. Redundant data was characterized as being unacceptable in many cases, but appropriate in others. For you to better understand how to determine when redundant data is acceptable, a definition of the term is in order.

Redundant data is a value that is repeated in a field as a result of the field's participation in relating two tables or as a result of some field or table anomaly. In the first instance, the redundant data is appropriate; by definition, a field used to relate one table to another will contain redundant data. (You'll learn more about this in Chapter 10.) Redundant data is entirely unacceptable in the second instance, however, because it poses problems with data consistency and data integrity; therefore, you should always strive to keep redundant data to an absolute minimum.

A *duplicate field* is a field that appears in two or more tables for any of these reasons.

- It is used to relate a set of tables together.
- It indicates multiple occurrences of a particular type of value.
- There is a perceived need for supplemental information.

The only instance in which a duplicate field is necessary is when it serves to establish a relationship between two tables; it provides the sole means of associating records in the first table with records in the second table. Duplicate fields are unnecessary in all other cases, and you should avoid them because they introduce needless, redundant data.

As you refine each table structure, you'll assess whether to retain a given duplicate field in the table. If the reason for its existence in the table is valid, then you'll keep it; otherwise, you'll remove it. You'll learn how to deal effectively with both redundant data and unnecessary duplicate fields in the following sections.

Using an Ideal Table to Refine Table Structures

Despite your efforts to refine the fields in a table, the table structure itself may contain anomalies that can produce unnecessary redundant data and make working with the data in the table difficult. You can identify a potentially problematic table structure by determining whether it complies with the Elements of the Ideal Table.

Elements of the Ideal Table

The Elements of the Ideal Table constitute a set of guidelines you can use to create sound table structures and to spot poorly designed tables easily. An ideal table structure has the following characteristics:

- *It represents a single subject, which can be an object or event.* Yes, I know, I've said this a number of times already. The fact of the matter is that I can't overemphasize this point. As long as you guarantee that each of your tables represents a single subject, you greatly reduce the risk of potential data integrity problems. This element validates the work you've done during the analysis and interview stages of the database design process, as well as the work you've just recently performed.

- *It has a primary key.* This is important for two reasons: A primary key uniquely identifies each record within a table, and it plays a key role (no pun intended) in establishing table relationships. Additionally, it has specific characteristics that help to implement and enforce various levels of data integrity. If you fail to assign a primary key to each table, you will eventually have data integrity problems. Chapter 8, “Keys,” covers primary keys in greater detail.
- *It does not contain multipart or multivalued fields.* Theoretically, you should have resolved these issues when you refined the field structures. Nevertheless, it’s still a good idea to review the fields one last time to ensure that you’ve completely removed each and every one of them.
- *It does not contain calculated fields.* Although you might believe that your current table structures are free of calculated fields, you may have accidentally overlooked one or two calculated fields during the field refinement process. This is a good time to review the table structures once more and make certain you remove those calculated fields you may have missed.
- *It does not contain unnecessary duplicate fields.* (Note that this guideline does not apply to fields used to relate a set of tables together, such as those used in the example in Figure 7.22.) One of the hallmarks of a poorly designed table is the inclusion of duplicate fields from other tables. You might feel compelled to add duplicate fields to a table for one of two reasons: to provide reference information or to indicate multiple occurrences of a particular type of value. Duplicate fields such as these raise various difficulties when you work with the data or attempt to retrieve information from the table.
- *It contains only an absolute minimum amount of redundant data.* Remember that a relational database will never be completely free of redundant data. However, you can—and should—make certain that each table contains as little redundant data as possible.

Resolving Unnecessary Duplicate Fields

Before you make final modifications to the table structures, you must first remove all unnecessary duplicate fields from the database. You can then refine the tables so that they comply with the Elements of the Ideal Table.

Duplicate fields that serve to provide reference information (also known as *reference fields*) are unnecessary and easy to resolve—you just remove them from the table. Unfortunately, many people believe that a table must contain every field that will appear in the reports they generate from it, so they introduce into the table various duplicate fields they deem are necessary. They assume that the table will then be able to provide all the requisite information for their reports. However, they are mistaken, and their action is both unwise and undesirable. Tables containing reference fields exhibit poor design and will have a number of problems, many of which will become increasingly clear as the database design process unfolds. Reference fields force the user or database application program to ensure that the values in all occurrences of the field are mutually consistent, a process that carries a high risk of error. Figure 7.23 shows an example of a table containing reference fields.

The `MANPHONE` and `WEB SITE` fields in the `INSTRUMENTS` table are reference fields and, by definition, are actually unnecessary duplicate fields. You certainly don't need to include them in this table because they're already part of the `MANUFACTURERS` table structure; therefore, you can remove them from the `INSTRUMENTS` table to resolve the unnecessary duplication problem. (`MANUFACTURER` is not a reference field because it currently relates the `INSTRUMENTS` table to the `MANUFACTURERS` table.) You'll learn in Chapter 12, "Views," that you can work with fields from the `INSTRUMENTS` table *and* the `MANUFACTURERS` table at the same time by combining them within a *view* (virtual table). You can then use this view as the basis for compiling any reports you require.

These fields duplicate the ManPhone and Web Site fields in the MANUFACTURERS table.



Instruments

Instrument ID	Instrument Description	Category	Price	Manufacturer	ManPhone	Web Site
2201	Stratocaster	Guitar	\$ 799.99	Fender Musical Instruments	481 596-9690	www.fender.com
3349	Player 2100 Multi-Effects	Multi-Effect Unit	\$ 174.99	Samson Technologies Corp.	322 364-2244	www.samsontech.com
1001	JCM 2000 Tube Super Lead	Amplifier	\$ 999.99	Mesa/Boogie	230 778-6565	www.mesaboogie.com
5590	Crate VC60 Pro Tube Amp	Amplifier	\$ 879.99	St. Louis Music, Inc.	324 738-7563	www.crateamps.com
2227	Cry Baby Wah-Wah	Single-Effect Unit	\$ 169.99	Dunlop Manufacturing, Inc.	230 745-2722	www.jimdunlop.com
2766	Twin Reverb Reissue	Amplifier	\$ 1,449.99	Fender Musical Instruments	481 596-9690	www.fender.com

Manufacturers

Manufacturer	ManStreet Address	ManCity	ManState	ManZipcode	ManPhone	Web Site
Dunlop Manufacturing, Inc.	PO Box 846	Benicia	CA	94510	230 745-2722	www.jimdunlop.com
Fender Musical Instruments	17600 N. Perimeter Drive	Scottsdale	AZ	85225	481 596-9690	www.fender.com
Mesa/Boogie	7426 Sunset Blvd.	Hollywood	CA	90046	230 778-6565	www.mesaboogie.com
Samson Technologies Corp.	PO Box 9031	Syosset	NY	11791	322 364-2244	www.samsontech.com
St. Louis Music, Inc.	1400 Ferguson Avenue	St. Louis	MO	63133	324 738-7563	www.crateamps.com

Figure 7.23 Example of a table containing reference fields.

Duplicate fields that serve to indicate multiple occurrences of the same type of value are unnecessary as well. For example, look at the version of the STUDENTS table presented in Figure 7.24.

Students

StdFirst Name	StdLast Name	StdStreet Address	<< other fields >>	Instrument 1	Instrument 2	Instrument 3
Scott	Baker	2904 Madison Ave		Guitar	Tenor Sax	
Michael	Chow	7410 Taxco Drive		Tenor Sax	Clarinet	Electric Piano
Debbie	McGuire	332 158th Ave SE		Drum Set	Bass Guitar	
Angie	Thomson	970 Pine Blvd		Guitar	Electric Piano	Snare Drum

These duplicate fields represent three occurrences of the same *type* of value.

Figure 7.24 A simple example of a table containing unnecessary duplicate fields.

INSTRUMENT 1, INSTRUMENT 2, and INSTRUMENT 3 are duplicate fields that represent multiple occurrences of the *same type* of value. Their purpose in the table is to enable the music department to keep track of the instruments checked out by a given student. Aside from the difficulties these fields pose in retrieving information about a particular instrument, the fields also limit the number of instruments a student can check out. What happens if several students want to check out more than three instruments?

Does this type of field structure look strangely familiar? It should! It's similar to the one back in Figure 7.18. As you've probably already guessed, it's nothing more than a flattened multivalued field. Mind you, the person who created this table probably didn't have a multivalued field in mind (and neither do most folks who create fields such as these), but that is what it truly is.

You already know how to deal with these unnecessary duplicate fields because you know how to resolve multivalued fields. You can easily fix the STUDENTS table by first visualizing the INSTRUMENT 1, INSTRUMENT 2, and INSTRUMENT 3 fields as a singular multivalued field, and then resolving it as you would any multivalued field. Figure 7.25 illustrates

this process. The shaded version of the STUDENTS table shows how you visualize the instrument fields as a singular multivalued field. You then resolve the multivalued field by applying the three-step process you learned earlier, which yields the revised STUDENTS table and the new STUDENT INSTRUMENTS table. When you're finished, you'll be able to enter any number of instruments for a particular student. You can then easily retrieve information such as the names of the students who have checked out a guitar, a list of the instruments that are currently checked out by a particular student, and the number of students who have checked out an electric piano.

Students				
StdFirst Name	StdLast Name	StdStreet Address	<< other fields >>	Instruments
Scott	Baker	2904 Madison Ave		Guitar, Tenor Sax
Michael	Chow	7410 Taxco Drive		Tenor Sax, Clarinet, Electric Piano
Debbie	McGuire	332 158th Ave SE		Drum Set, Bass Guitar
Angie	Thomson	970 Pine Blvd		Guitar, Electric Piano, Snare Durm

Students				Student Instruments		
StdFirst Name	StdLast Name	StdStreet Address	<< other fields >>	StdFirst Name	StdLast Name	Instrument
Scott	Baker	2904 Madison Ave		Scott	Baker	Guitar
Michael	Chow	7410 Taxco Drive		Scott	Baker	Tenor Sax
Debbie	McGuire	332 158th Ave SE		Michael	Chow	Tenor Sax
Angie	Thomson	970 Pine Blvd		Michael	Chow	Clarinet
				Michael	Chow	Electric Piano
				Debbie	McGuire	Drum Set
				Debbie	McGuire	Bass Guitar

Figure 7.25 Resolving a simple set of unnecessary duplicate fields.

In some instances, a table can contain two or more sets of duplicate fields that represent multiple occurrences of the same type of value. Figure 7.26 shows a slightly different version of the STUDENTS table

shown in Figure 7.24; this version contains *two* sets of duplicate fields. You may be thinking at this very moment, “Why is he saying there are *two* sets of duplicate fields when I clearly see *three*?” Contrary to what you may think, INSTRUMENT 1/CHECKOUT DATE 1, for example, does not constitute a set of duplicate fields. Quite the opposite—INSTRUMENT 1/INSTRUMENT 2/INSTRUMENT 3 constitute the first set of duplicate fields, and CHECKOUT DATE 1/CHECKOUT DATE 2/CHECKOUT DATE 3 constitute the second set of duplicate fields.

Students

StdFirst Name	StdLast Name	<< other fields >>	Instrument 1	Checkout Date 1	Instrument 2	Checkout Date 2	Instrument 3	Checkout Date 3
Scott	Baker		Guitar	09/26/19	Tenor Sax	09/28/19		
Michael	Chow		Tenor Sax	09/26/19	Clarinet	10/03/19	Electric Piano	10/16/19
Debbie	McGuire		Drum Set	11/14/19	Bass Guitar	11/20/19		
Angie	Thomson		Guitar	11/14/19	Electric Piano	11/14/19	Snare Drum	12/05/19

Figure 7.26 Example of a table with multiple sets of duplicate fields.

You’ve probably realized that these two sets of duplicate fields are actually two flattened multivalued fields and that you can resolve them in the same manner as in the previous example. The only other issue that you must be concerned with is the distinct one-to-one association between an instrument and a checkout date. This won’t be a problem, however, because you’ve dealt with this type of scenario before. If you visualize one multivalued field called INSTRUMENTS and another called CHECKOUT DATE, you’ll see that the overall table structure is quite similar to the one in Figure 7.21. (In that figure, there’s a one-to-one association between the CATEGORIES TAUGHT and MAXIMUM LEVEL TAUGHT fields.)

Figure 7.27 illustrates how you can fix this table. As before, the shaded version of the STUDENTS table shows how you visualize the instrument and checkout date fields as singular multivalued fields. You then resolve the multivalued fields by applying the three-step process you learned earlier, yielding the revised STUDENTS table and the new STUDENT INSTRUMENTS table.

Students				
StdFirst Name	StdLast Name	<< other fields >>	Instruments	Checkout Dates
Scott	Baker		Guitar, Tenor Sax	09/26/19, 09/28/19
Michael	Chow		Tenor Sax, Clarinet, Electric Piano	09/28/19, 10/03/19, 10/16/19
Debbie	McGuire		Drum Set, Bass Guitar	11/14/19, 11/20/19
Angie	Thomson		Guitar, Electric Piano, Snare Durm	11/14/19, 11/14/19, 12/05/19

Students				Student Instruments			
StdFirst Name	StdLast Name	StdStreet Address	<< other fields >>	StdFirst Name	StdLast Name	Instrument	Checkout Date
Scott	Baker	2904 Madison Ave		Scott	Baker	Guitar	09/26/19
Michael	Chow	7410 Taxco Drive		Scott	Baker	Tenor Sax	09/28/19
Debbie	McGuire	332 158th Ave SE		Michael	Chow	Tenor Sax	09/28/19
Angie	Thomson	970 Pine Blvd		Michael	Chow	Clarinet	10/03/19
				Michael	Chow	Electric Piano	10/16/19
				Debbie	McGuire	Drum Set	11/14/19
				Debbie	McGuire	Bass Guitar	11/20/19

Figure 7.27 Resolving the multiple sets of duplicate fields in the STUDENTS table.

Now that you’re familiar with the Elements of the Ideal Table, review your table structures and refine them as necessary. When you’re in doubt about a particular table, sketch its structure on a piece of paper and load it with sample data. You’ll then be able to resolve the anomalies revealed by the data.

Establishing Subset Tables

As you refine the structures of your tables, you may find that some of the fields in a particular table do not always contain values. This situation will not affect your ability to retrieve information from the table, but it can indicate that the table might need further refinement. Consider the structure of the INVENTORY table in Figure 7.28.

Table Structures	
Inventory	
Item Name	Model
Item Description	Warranty Expiration Date
Current Value	Publisher
Insured Value	Author
Date Entered	ISBN
Manufacturer	Category

Figure 7.28 Structure of an office inventory table.

In this scenario, the table contains data about various items in a person's office, such as office furniture, office equipment (computers, printers, and so forth), and books. It's inevitable that the values of several fields in many of the records will be blank. For example, a book will not have a MANUFACTURER, MODEL, or WARRANTY EXPIRATION DATE, and a printer will not have an AUTHOR, PUBLISHER, ISBN, or CATEGORY. This doesn't pose a problem from a *physical* viewpoint (limited hard-disk space certainly isn't the critical issue it was in years past), but it can pose a *perceptual* problem. Users (and management, for that matter) get fairly nervous when they see a lot of blank values in a table. Is the data missing? Did someone forget to make entries into these fields? Has someone mistakenly deleted the data? Did the computer accidentally destroy the original values? (Yes, the urban myth, "The computer did it!" still lives on.) The more important question is this: If you were adhering to the Elements of the Ideal Table as you were creating this table, how did you arrive at this particular structure?

Fortunately, this is just another type of structural anomaly that occasionally occurs as you design various tables. Your task now is to learn how to deal with it in an appropriate manner.

The first step is to determine whether the INVENTORY table truly complies with the first element of an ideal table (that is, “It represents a single subject”). A table that contains a large number of blank values in its fields usually—but not always—represents more than one subject. Think about the two sets of fields in question for a moment, and you’ll soon realize that they represent characteristics of *two distinct aspects* of the table’s subject. The first set of fields describes equipment inventory, and the second set of fields describes books inventory; furthermore, both types of inventory share common characteristics, such as ITEM NAME, ITEM DESCRIPTION, and CURRENT VALUE. In essence, “Equipment” and “Books” are subjects that are dependent upon the INVENTORY table for their very existence; neither describes a completely distinct object or event. As a result, they are *subordinate subjects*, and you’ll create a *subset* table for each of them.

Just as a data table represents a distinct subject, a subset table represents a subordinate subject of a particular data table. The subset table contains fields that are germane to the subordinate subject it represents, and it also includes a field (or fields) from the data table that serves to relate the data table to the subset table. It’s important to note that a subset table *does not* contain fields that represent characteristics common to both it and the data table; these fields must remain in the data table.

Now that you’ve determined that the INVENTORY table describes three subjects (it doesn’t matter that two of them are subordinate subjects), you must bring it into compliance with the first element of an ideal table by removing the fields in question. You then use the fields as the basis for two new subset tables, one for each subordinate subject. Here are the steps you follow to accomplish these tasks.

1. Use the MANUFACTURER, MODEL, and WARRANTY EXPIRATION DATE fields to create a new subset table called EQUIPMENT.

2. Use the PUBLISHER, AUTHOR, ISBN, and CATEGORY fields to create a new subset table called BOOKS.
3. Add ITEM NAME to both tables; this field will relate each subset table to the data table.
4. Compose a suitable description for both subset tables and add them to the Final Table List. Indicate each table's type as "Subset."

Figure 7.29 shows the new subset table structures.

Table Structures		
Inventory	Equipment	Books
Item Name	Item Name	Item Name
Item Description	Manufacturer	Publisher
Current Value	Model	Author
Insured Value	Warranty Expiration Date	ISBN
Date Entered		Category

Figure 7.29 *The new subset table structures.*

Take a moment to review your table structures once more. You may discover that you've created subset tables without knowing it. Tables that have *almost* identical structures are commonly subset tables; there are usually only a few unique fields that distinguish one table from the other. For example, consider the two partial table structures in Figure 7.30. Each table represents a distinct aspect of the *same* subject.

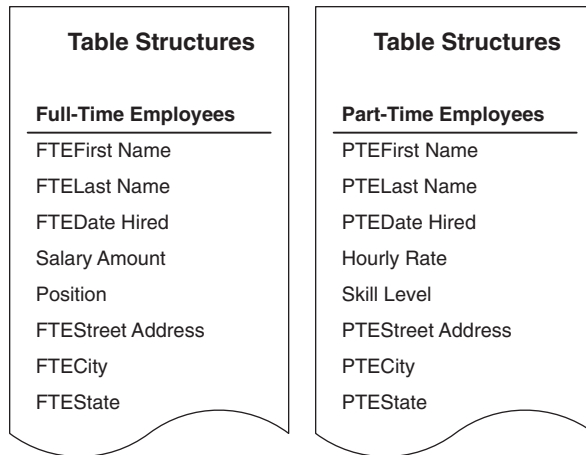


Figure 7.30 *Previously unidentified subset tables.*

Both of these tables represent employees, but each represents a *specific type* of employee. Notice, however, that there are generic fields common to both tables: first name, last name, date hired, street address, city, and state. These fields are duplicated unnecessarily, so you'll need to refine the table structures to resolve this problem.

Refining Previously Unidentified Subset Tables

When you identify subset tables such as these, you can refine them using these steps.

1. Remove all the fields that the subset tables have in common and use them as the basis for a new *data* table.
2. Identify what subject the new data table represents, and then give that table an appropriate name.
3. Make sure that the subset tables represent subordinate subjects of the data table and modify the subset table names as necessary.
4. Compose a suitable description for the data table and then add it to the Final Table List. Indicate the table type as "Data."

Figure 7.31 shows the results of using these steps on the FULL-TIME EMPLOYEES and PART-TIME EMPLOYEES tables.

Table Structures		
Employees	Full-Time Employees	Part-Time Employees
EmpFirst Name	EmpFirst Name	EmpFirst Name
EmpLast Name	EmpLast Name	EmpLast Name
Date Hired	Salary Amount	Hourly Rate
EmpStreet Address	Position	Skill Level
EmpCity		
EmpState		

Figure 7.31 The results of refining the subset tables.

At this point, all of your table structures should be in pretty good shape. You will need to refine them even further, however, as you learn about primary keys, foreign keys, relationships, and business rules.

EXAMPLE: ESTABLISHING TABLE STRUCTURES

You're now going to define the *Preliminary Table List* for Mike's Bikes. As you know, the first thing you need to do is review the Preliminary Field List to determine what subjects you can infer from the fields on the list. Figure 7.32 shows a *partial* sample of that list.

After carefully reviewing the entire Preliminary Field List, you determine that the fields on the list suggest these subjects: customers, employees, invoices, products, and vendors. You then compile these items into the first version of your *Preliminary Table List*.

Preliminary Field List	
<i>as of 02/16/19</i>	
Birth Date	Office Phone
Employee Name	Product Name
Employee Address	Category
Employee City	Unit Price
Customer Name	Invoice Number
Customer Address	Invoice Date

Figure 7.32 *The Preliminary Field List for Mike's Bikes.*

Now you create a *second* version of the list by merging the current Preliminary Table List with the *list of subjects* you created during the analysis process. Keep the following steps in mind as you merge the two lists together.

1. *Resolve items that are duplicated on both lists.* Remember that a single item can appear on both lists, yet represent different subjects. When you identify such items, use the appropriate techniques to resolve this problem.
2. *Resolve items that represent the same subject but have different names.* You want to ensure that only one table represents a particular subject.
3. *Combine the remaining items together into one list.* The combined list becomes the second version of the Preliminary Table List.

After following these steps, your Preliminary Table List should look similar to the one shown in Figure 7.33.

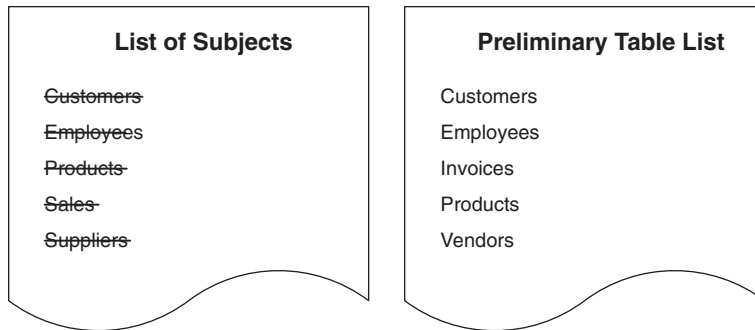


Figure 7.33 The second version of the Preliminary Table List.

You cross out “Customers,” “Employees,” and “Products” on the list of subjects because they represent the same subjects as their counterparts on the Preliminary Table List. The SALES table has no counterpart on the Preliminary Table List, but it does represent the *same* subject as “Invoices.” “Invoices” is most meaningful to Mike and his staff, however, so you use it on the Preliminary Table List instead of “Sales.” A similar situation exists between “Suppliers” and “Vendors”; Mike selects “Vendors” as the name to appear on the Preliminary Table List, so you cross out “Suppliers.”

❖ **Note** Selecting a name that best represents the subject of the table is an arbitrary task. A good rule to follow is to use the name that is *most* meaningful to everyone in the organization.

Now you’ll work toward the final version of the Preliminary Table List. Use the mission objectives you created at the beginning of the database design process to determine whether there are subjects you may have overlooked during the previous two procedures. Identify each subject represented in the mission objectives using the Subject-Identification Technique. After you’ve identified as many subjects as possible, you can use the steps from the second procedure to cross-check these subjects against the subjects currently listed on the Preliminary Table List. When you’ve completed the review and have

resolved any duplicate items, your final version of the Preliminary Table List is complete.

As it turns out, *all* the subjects you've identified from the mission objectives for Mike's Bikes already appear on the Preliminary Table List. This is good news because it allows you to complete your cross-check quite easily. Satisfied that you've completed the task thoroughly, you now have the final version of the Preliminary Table List.

Now that the Preliminary Table List is complete, you're ready to transform it into a *Final Table List*. Keep these steps in mind as you begin this process.

1. *Refine the table names.* Use the appropriate guidelines to ensure that each table name is clear, unambiguous, descriptive, and meaningful.
2. *Compose a suitable description for each table.* Make certain that the table description explicitly defines the table and states its importance to the organization. Use the pertinent guidelines to create each table description.
3. *Indicate the table's type.* Remember that a table can be classified in one of four ways: *data*, *linking*, *subset*, or *validation*. At this point, all of your tables are *data* tables.

Figure 7.34 shows a *partial* example of the Final Table List for Mike's Bikes.

The next order of business is to associate fields from the Preliminary Field List with each table in the Final Table List. Make certain you select the fields that best represent characteristics of each table's subject; each field should define or describe a particular aspect of the subject. Figure 7.35 shows a *partial* example of the table structures for Mike's Bikes.

Final Table List		
Name	Type	Description
Customers	Data	The people who purchase the products we have to offer. Keeping track of our customers allows us to promote our business and obtain valuable feedback in assessing the quality of our customer service.
Employees	Data	The people who work for our company in various capacities. This information is important for tax purposes, health benefits, and work-related issues.

Figure 7.34 A partial listing of the Final Table List for Mike’s Bikes.

Table Structures			
Customers	Employees	Invoices	Products
Customer First Name	Employee Name	Invoice Number	Product Name
Customer Last Name	Employee Address	Invoice Date	Product Description
Customer Phone	Employee Phone	Employee Name	Category
Customer Address	SSN	Customer Last Name	Wholesale Price
Status	Date Hired	Customer First Name	Retail Price
	Position	Customer Phone	Quantity

Figure 7.35 A partial listing of the table structures for Mike’s Bikes.

Now you refine the fields. Remember to follow these steps as you work with each field.

Improve the field name. Use the appropriate guidelines to ensure that each field name is as clear, unambiguous, and descriptive as possible.

Determine whether the field complies with the Elements of the Ideal Field. Make certain you check for multipart and multivalued fields. As you learned earlier, they can cause a number of problems within a table.

As you review the fields, you decide to abbreviate some of the field names in the CUSTOMERS, EMPLOYEES, and INVOICES tables, shortening CUSTOMER to CUST and EMPLOYEE to EMP. You also decide that the field name QUANTITY (in the PRODUCTS table) does not completely describe the characteristic it represents, so you change it to QUANTITY ON HAND. The phone fields in the CUSTOMERS and EMPLOYEES tables suffer the same problem, so you change them to CUSTHOME PHONE and EMPHOME PHONE, respectively. Furthermore, you change SSN to SOCIAL SECURITY NUMBER so that the field name is absolutely unambiguous.

Further investigation of the fields reveals that almost all of them comply with the Elements of the Ideal Field. The only exceptions are the address fields in the CUSTOMERS and EMPLOYEES tables, and the EMPLOYEE NAME fields in the EMPLOYEES and INVOICES tables. After ascertaining that you can decompose each address field into four individual items—street address, city, state, and ZIP code—you transform these items into fields and add them to the CUSTOMERS and EMPLOYEES tables. Similarly, you notice that the EMPLOYEE NAME field represents two items—first name and last name—and you make the appropriate adjustments to that field in the EMPLOYEES and INVOICES tables.

Figure 7.36 shows the result of all the changes you've made to the fields.

Table Structures			
Customers	Employees	Invoices	Products
CustFirst Name	EmpFirst Name	Invoice Number	Product Name
CustLast Name	EmpLast Name	Invoice Date	Product Description
CustHome Phone	EmpHome Phone	EmpFirst Name	Category
CustStreet Address	Social Security Number	EmpLast Name	Wholesale Price
CustCity	EmpStreet Address	CustFirst Name	Retail Price
CustState	EmpCity	CustLast Name	Quantity On Hand
CustZipcode	EmpState	CustHome Phone	

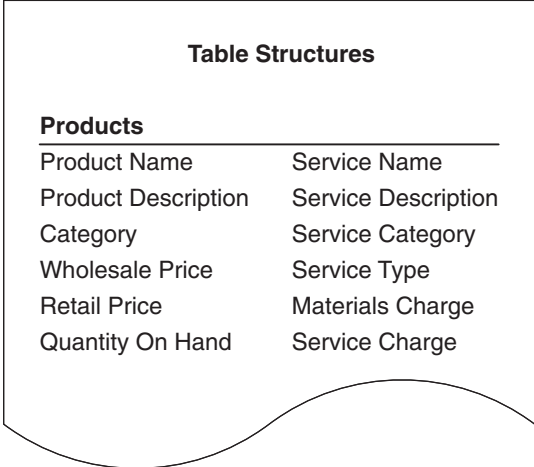
Figure 7.36 Refinements to the fields in the table structures.

Your final task is to refine the table structures. Make certain that you have assigned the appropriate fields to each table and that you have properly defined each table. Remember to follow these steps as you work with each table.

1. *Resolve unnecessary duplicate fields.* When you create new tables as a result of resolving duplicate fields, make sure you properly identify them and add them to the Final Table List.
2. *Determine whether each table complies with the Elements of the Ideal Table.* Make certain you resolve all the anomalies you identify in the fields or within the table structure as a whole.
3. *Establish subset tables as appropriate.* Make certain you properly identify these tables and add them to the Final Table List as well.

As you complete your review of the tables, you determine that all of them conform to the Elements of the Ideal Table with the exception of the INVOICES table. The only problem with this table is that it contains an unnecessary duplicate field: CUSTHOME PHONE. You can remove this field from the table, however, because it provides only reference information.

As you work with the PRODUCTS table, you notice that there are fields you might be able to remove and then use as the basis for a subset table. So you review the table once again. Figure 7.37 shows the PRODUCTS table structure you're currently examining. (This is an expanded version of the table structure shown in Figure 7.36.)



Products	Service Name
Product Name	Service Name
Product Description	Service Description
Category	Service Category
Wholesale Price	Service Type
Retail Price	Materials Charge
Quantity On Hand	Service Charge

Figure 7.37 *The PRODUCTS table structure (expanded version).*

Your assumption proves correct. You determine that certain fields describe a service, and you can construe a service as being a different type of product. A service is similar to a product in that it has a name, description, and category, but it is different inasmuch as it has a type, materials charge, service charge, and service date. With this in mind, you create a new subset table called SERVICES, make the appropriate modifications to the PRODUCTS table, and use the PRODUCT NAME field

to relate the two tables to each other. You then add the suitable listing for the SERVICES table to the Final Table List. Figure 7.38 shows the revised PRODUCTS table and the new SERVICES subset table.

Table Structures	
Products	Services
Product Name	Product Name
Product Description	Service Type
Category	Materials Charge
Wholesale Price	Service Charge
Retail Price	
Quantity On Hand	

Figure 7.38 The new PRODUCTS and SERVICES tables.

Summary

We opened the chapter with a discussion of the *Preliminary Table List*. This list constitutes the initial table structures for the new database. You learned how to develop this list using the Preliminary Field List, the list of subjects, and the mission objectives, all of which you compiled during the analysis phase of the database design process.

Next we discussed the procedure for transforming the Preliminary Table List into a *Final Table List*, which contains the name, type, and description of each table in the database. You learned a set of *guidelines for creating table names*, and another set of *guidelines for composing table descriptions*. We then worked on creating table names that are unambiguous, descriptive, and meaningful and descriptions that explicitly define tables, as well as stating their importance to the organization. You also learned that enlisting the help of users and

management is crucial to the process of developing well-defined table descriptions. Table descriptions must be suitable and easily understood by everyone in the organization.

We then discussed the process of associating fields with each table on the Final Table List. Here you learned how to build a structure for a given table using fields from the Preliminary Field List that best represent characteristics of the table's subject.

Refining fields was the next subject of discussion, and you learned a set of guidelines for creating field names that will help you ensure that they are clear, descriptive, and meaningful. You also learned about the Elements of the Ideal Field. Now you know that you can resolve anomalies in a field by determining whether it complies with these elements. We then discussed how to resolve *multipart* and *multivalued* fields. You learned that decomposing multipart fields yields new fields, whereas decomposing multivalued fields yields new tables.

The chapter closed with a discussion of refining table structures. You learned to identify the Elements of the Ideal Table, and you now know that you can ferret out a problem in table structure by determining whether a table complies with these elements. We then discussed *unnecessary duplicate fields*, and you now know that they appear in a table for one of two reasons: to supply reference information or to represent different occurrences of the same type of value. You then learned how to resolve duplicate fields to eliminate the problems they present.

The final discussion centered on the topic of *subset tables*. As you now know, a subset table represents a *subordinate subject* of a particular data table, and a distinct relationship exists between the subset table and the data table. You also know that you can explicitly create subset tables. You then learned that you may have unknowingly created subset tables earlier in the database design process and that you need to look for subset tables you have not previously identified. When you identify a subset table, you refine it and add it to the Final Table List.

Review Questions

1. How do you identify and establish tables for a new database?
2. Why do you use the Preliminary Field List to help you define tables for the database?
3. What action do you take when an item on the list of subjects and a differently named item on the Preliminary Table List both represent the same subject?
4. What information does the *Final Table List* provide?
5. State three guidelines for creating table names.
6. State two guidelines for composing table descriptions.
7. How do you assign fields to a table on the *Final Table List*?
8. State three guidelines for creating field names.
9. What two problems can poorly designed fields cause?
10. What can you use to resolve field anomalies?
11. State three of the Elements of the Ideal Field.
12. Under what condition is redundant data acceptable?
13. In general terms, what three steps do you follow to resolve a multi-valued field?
14. When is it necessary to use a duplicate field in a table?
15. How can you refine table structures?
16. State three of the Elements of the Ideal Table.
17. What is a *subset table*?